

\$ Aide-mémoire Docker

Les commandes essentielles, groupées par usage. Chaque partie d'une commande garde toujours la même couleur pour qu'on la repère d'un coup d'œil.

■ commande racine ■ sous-commande ■ option / flag ■ valeur à remplacer

Conteneurs et images — les bases

```
docker ps
```

Liste les conteneurs actifs.

```
docker ps -a
```

Liste **tous** les conteneurs, y compris ceux arrêtés (utile pour les redémarrer).

```
docker start <container>
```

Démarre un conteneur arrêté.

```
docker stop <container>
```

Arrête un conteneur en cours d'exécution.

```
docker rm <container>
```

Supprime un conteneur.

```
docker exec -it <container> bash
```

Ouvrir un terminal interactif à l'intérieur d'un conteneur déjà en cours d'exécution (-i = interactif, -t = alloue un pseudo-terminal). Utile pour inspecter le conteneur ou y lancer une commande ponctuelle.

```
docker inspect <container> | less
```

Affiche tous les détails d'un conteneur au format JSON (configuration, réseau, montages, variables d'environnement...). Fonctionne aussi sur une image, un volume ou un réseau. | `less` permet de parcourir une sortie longue page par page.

```
docker exec -it mysql-docker-container bash # 1. entre dans le conteneur
mysql -u root -p # 2. lance le client MySQL (demande le mot de
passe)
show databases; # 3. liste les bases présentes
exit # 4. quitte le client mysql
exit # 5. quitte le shell bash du conteneur
```

Exemple complet : se connecter à une base MySQL qui tourne dans un conteneur, l'interroger, puis ressortir proprement.

```
docker run <image>
```

Crée un **nouveau conteneur** à partir de l'image et le démarre (télécharge l'image si elle n'est pas déjà en local).

```
docker images
```

Liste les images présentes en local.

```
docker rmi <image_ou_id>
```

Supprime une image.

```
docker pull <image>
```

Télécharge une image depuis un registre (ex. Docker Hub), sans créer de conteneur.

Options avancées de run

```
docker run -p 8082:8082 <image>
```

Publie un port : hôte:conteneur. Le port de gauche est celui exposé sur la machine hôte.

```
docker run --name <nom> <image>
```

Nomme le conteneur au lieu d'un nom aléatoire généré automatiquement. Ce nom sert ensuite directement pour stop, start et rm.

```
docker run -d <image>
```

Lance le conteneur en mode **détaché** (arrière-plan) : le terminal reste libre.

```
docker run -e VAR=valeur
```

Définit une variable d'environnement dans le conteneur. Répétable pour en passer plusieurs (ex. MYSQL_ROOT_PASSWORD, MYSQL_DATABASE, MYSQL_USER, MYSQL_PASSWORD pour une image MySQL).

```
docker run --network host
```

Fait partager au conteneur la pile réseau de la machine hôte : pas d'IP propre, les ports du conteneur sont directement ceux de l'hôte (rend -p inutile).

```
<image> --arg=valeur
```

Ce qui suit le nom de l'image est passé en argument à l'**application** dans le conteneur, pas à Docker. Ex. --spring.profiles.active=prod active un profil Spring Boot.

```
docker run -p 8082:8082 --name mon-conteneur -d mon-image:1.0.0
```

Exemple combinant les options ci-dessus.

```
docker run -e MYSQL_ROOT_PASSWORD=root_password -e MYSQL_DATABASE=my_database -e  
MYSQL_USER=app_user -e MYSQL_PASSWORD=app_password --network host mysql:latest
```

Exemple : lance MySQL avec ses identifiants et sa base initiale, en réseau hôte.

```
docker rm mon-service  
docker run --name mon-service -p 8082:8082 mon-service:2.0.0 --spring.profiles.active=prod
```

Exemple : remplace un conteneur existant par une nouvelle version de l'image, avec un profil Spring actif.

```
docker run -e SPRING_PROFILES_ACTIVE=<profil>
```

Variable d'environnement reconnue par Spring Boot : active un profil (ex. test, prod). Équivalent à passer --spring.profiles.active=<profil> en argument après l'image — deux façons de faire la même chose.

```
docker build -f Dockerfile-name -t mon-service:2.0.1 .  
docker run --name mon-service -e SPRING_PROFILES_ACTIVE=test -p 8082:8082 mon-service:2.0.1
```

Exemple : builder depuis un Dockerfile spécifique, puis lancer avec un profil de test.

```
docker run -e server.port=<port>
```

Redéfinit une propriété Spring Boot (ici server.port) via une variable d'environnement nommée exactement comme la propriété. Spring la reconnaît directement, sans passer par la forme habituelle SERVER_PORT. Le port choisi doit correspondre au port conteneur donné à -p.

```
docker run --name mon-service -e server.port=8081 -p 8081:8081 -e  
SPRING_PROFILES_ACTIVE=test mon-service:2.0.1
```

Exemple : port et profil définis via des variables d'environnement (le port doit être cohérent avec -p).

```
docker run --env-file <fichier>
```

Charge toutes les variables d'environnement depuis un fichier (une par ligne, format VAR=valeur), au lieu de les lister une par une avec plusieurs -e.

```
docker run --name mon-service -p 8081:8081 --env-file environment.env mon-service:2.0.1
```

Exemple : charger les variables depuis un fichier plutôt que de les passer une à une.

```
docker run --rm
```

Supprime automatiquement le conteneur dès qu'il s'arrête. Évite d'accumuler des conteneurs arrêtés, pratique pour un test ponctuel.

```
docker run --rm alpine
```

Exemple : lance un conteneur jetable, supprimé automatiquement à la sortie.

Build et gestion des images

```
docker build -t nom:version .
```

Construit une image depuis le Dockerfile du dossier courant, et la nomme (tag `nom:version`).

```
docker build -f <Dockerfile> -t nom:version .
```

Précise le fichier à utiliser comme Dockerfile (utile s'il y en a plusieurs, ex. un fichier `Dockerfile-name`). Sans `-f`, Docker cherche un fichier nommé `Dockerfile`.

```
docker image prune -a
```

Supprime toutes les images non utilisées par un conteneur en une seule fois.

Précision : sans `-a`, seules les images « dangling » (sans tag) sont supprimées. Avec `-a`, toute image non référencée par un conteneur part aussi — ce n'est pas littéralement « toutes les images » du système.

Filtrage et nettoyage

```
docker ps -a --filter "clé=valeur"
```

Liste les conteneurs en appliquant un filtre. Répéter `--filter` combine les filtres (ET logique).

```
docker ps -a --filter "ancestor=mon-service:2.0.0"
```

Filtre les conteneurs créés à partir de l'image `mon-service:2.0.0`.

```
docker ps -a --filter "status=exited"
```

Filtre les conteneurs arrêtés.

```
... -q | xargs docker rm
```

Ajouté à la suite d'un filtrage : supprime en une fois tous les conteneurs correspondants (`-q` ne renvoie que les IDs, `xargs` les passe un à un à `docker rm`).

Logs

```
docker logs <container>
```

Affiche les logs du conteneur.

```
docker logs -f <container>
```

Affiche les logs en continu (« follow »), utile pour suivre l'activité en direct.

Volumes

Un volume nommé permet de conserver les données d'un conteneur même s'il est supprimé (contrairement au système de fichiers du conteneur, qui disparaît avec lui). Docker le stocke sur l'hôte, dans `/var/lib/docker/volumes/<nom>/_data`.

```
docker volume create <nom>
```

Crée un volume nommé, géré par Docker.

```
docker run -v <volume>:<chemin_conteneur> <image>
```

Monte le volume à l'emplacement indiqué dans le conteneur.

```
docker volume ls
```

Liste tous les volumes existants sur la machine.

```
docker volume inspect <volume>
```

Affiche le détail d'un volume au format JSON : date de création, driver, point de montage sur l'hôte (Mountpoint), etc. Pour un volume anonyme, le label `com.docker.volume.anonymous` confirme qu'il n'a pas de nom choisi.

```
docker volume create app-data
docker run -v app-data:/my-app/data mon-service:2.0.1
```

Exemple : crée le volume `app-data` puis le monte dans le conteneur.

Volume anonyme

Si on ne précise que le chemin dans le conteneur (sans nom ni chemin hôte avant les deux-points), Docker crée quand même un volume — mais avec un identifiant généré automatiquement plutôt qu'un nom choisi. Il est donc plus difficile à retrouver et à réutiliser ensuite qu'un volume nommé.

```
docker run -v <chemin_conteneur> <image>
```

Crée un volume anonyme (nom généré par Docker) et le monte à ce chemin.

```
docker run -v /my-app/data mon-service:2.0.1
```

Exemple : Docker crée et gère seul le volume, sans nom choisi.

Bind mount

Autre façon de monter des données : au lieu d'un volume géré par Docker, on relie directement un dossier de la machine hôte au conteneur. C'est vous qui choisissez et gérez cet emplacement (pratique en développement pour accéder directement aux fichiers), contrairement au volume nommé dont l'emplacement est géré par Docker.

```
docker run -v <chemin_hôte>:<chemin_conteneur> <image>
```

Monte un dossier de l'hôte (chemin absolu ou relatif) directement dans le conteneur.

```
docker run -v ~/mon-app/data:/my-app/data mon-service:2.0.1
```

Exemple : monte le dossier ~/mon-app/data de l'hôte dans le conteneur.

tmpfs mount (Linux uniquement)

Monte un dossier directement en mémoire vive de l'hôte : rien n'est jamais écrit sur disque, les données disparaissent à l'arrêt du conteneur et ne sont partagées ni avec l'hôte ni avec d'autres conteneurs. Utile pour des fichiers temporaires ou sensibles (logs éphémères, caches).

```
docker run --tmpfs <chemin_conteneur> <image>
```

Monte le chemin donné en mémoire (tmpfs), Linux uniquement.

```
docker run --tmpfs /app/logs mon-service:2.0.1
```

Exemple : les logs de l'application vivent en mémoire, jamais sur disque.

Astuce : inspecter le contenu d'un volume

On peut monter un volume existant dans un conteneur jetable, juste pour regarder ce qu'il contient, sans toucher au conteneur qui l'utilise habituellement.

```
docker run --rm -it -v app-data:/data alpine sh
```

--rm supprime ce conteneur temporaire à la sortie, -it ouvre un shell interactif, et alpine est une image minimale pratique pour ce genre d'exploration ponctuelle. Une fois dedans, ls /data montre le contenu du volume.